

Content basics

Andres Ibanez de Aldecoa



Content Types

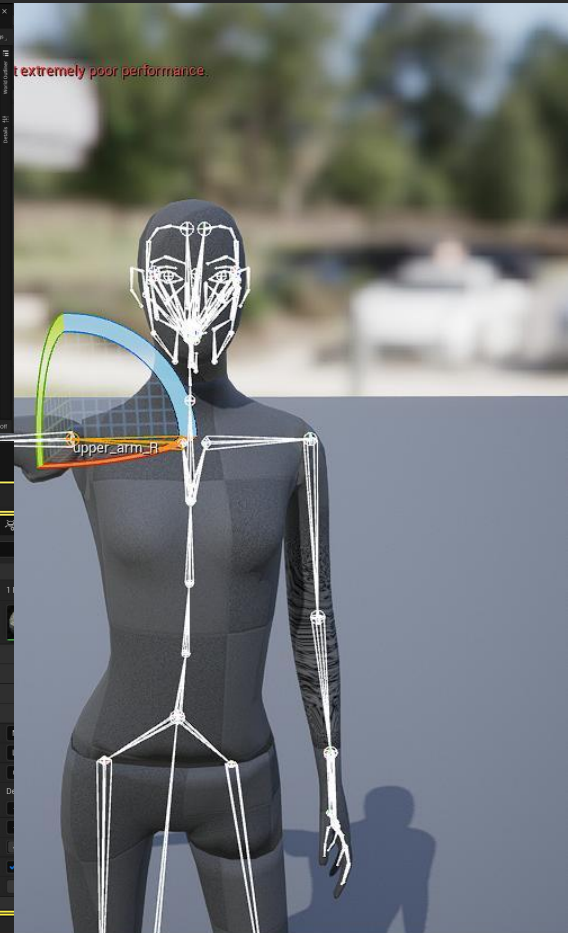
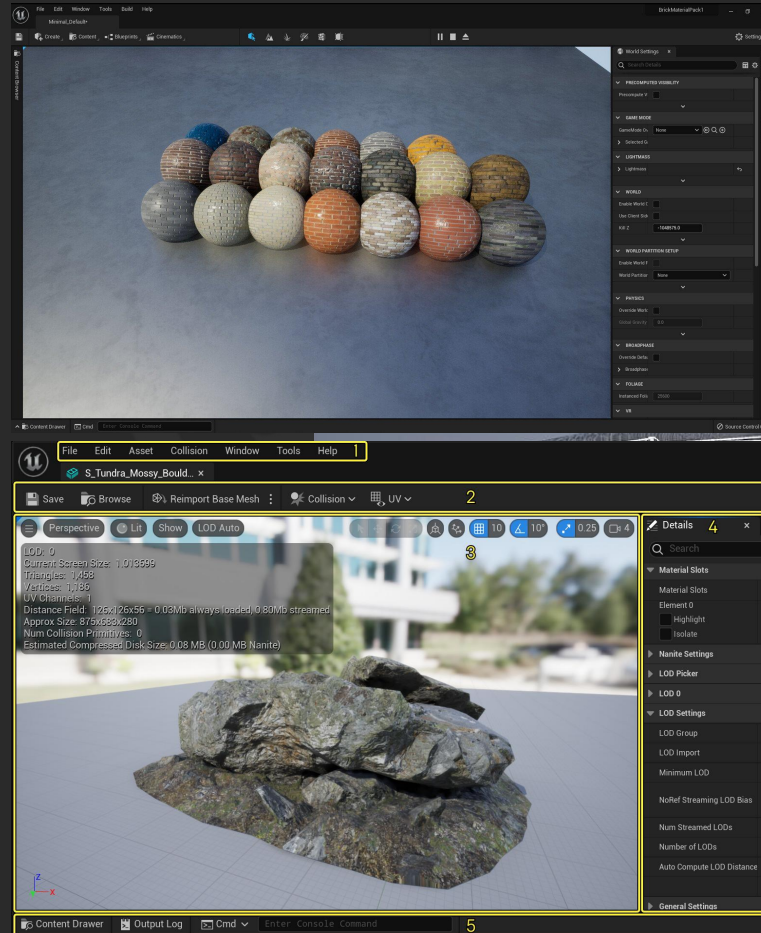
Materials

Static Mesh

Geometry Collections

Skeletal Mesh

Visual Effects



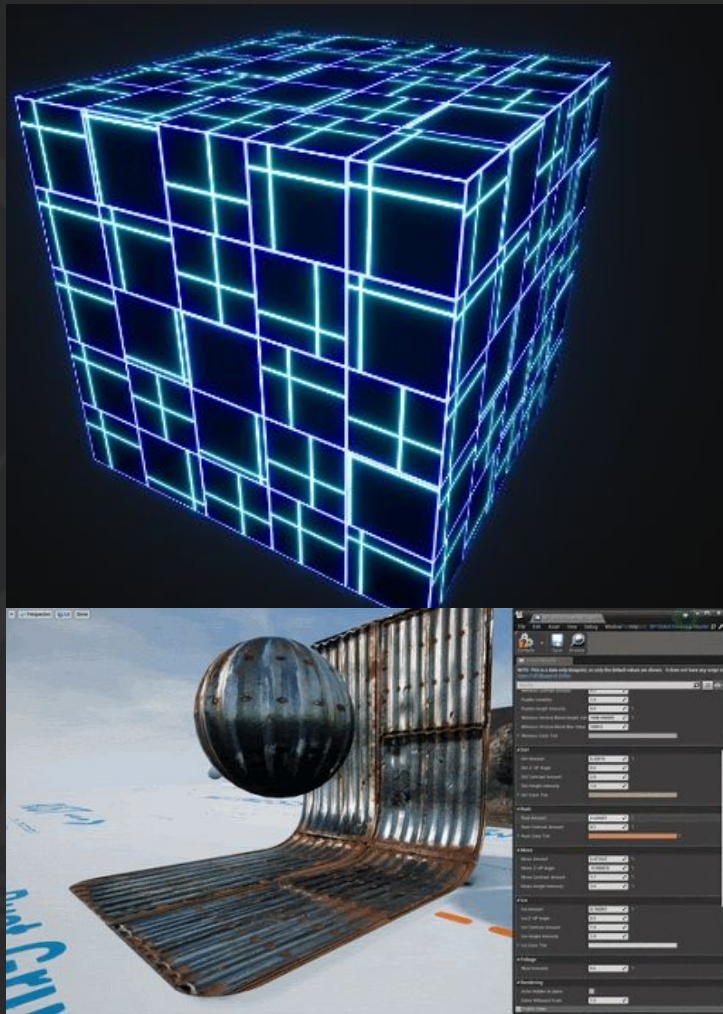
Materials

Materials are what give your world color or lack thereof.

Materials similar to blueprints have a variety amount of options to choose from and generate content with.

Materials are highly customizable and allow for a lot of possibilities both in gameplay and for visual effects.

“Behind the scenes, these node graphs are silently translated to HLSL. As a user, you can view the HLSL code but cannot edit it directly. This makes Material creation in Unreal Engine intuitive and approachable.”



Material Introduction

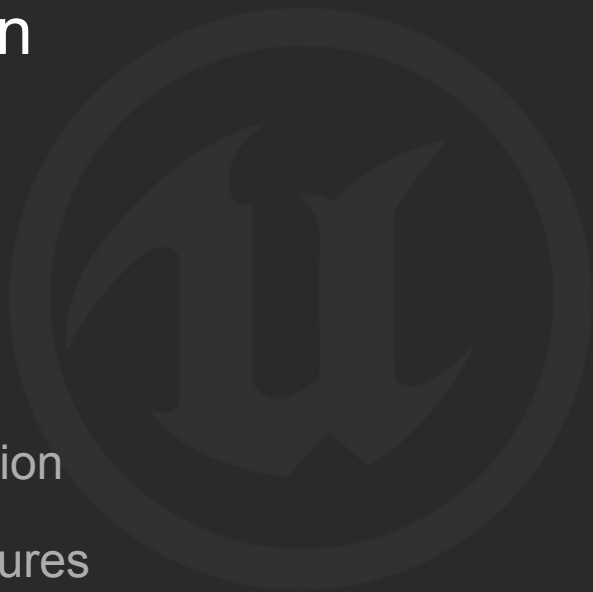
Creating a New Material

Material Properties

Material Expressions

Variables and Parametrization

Working with Data and textures



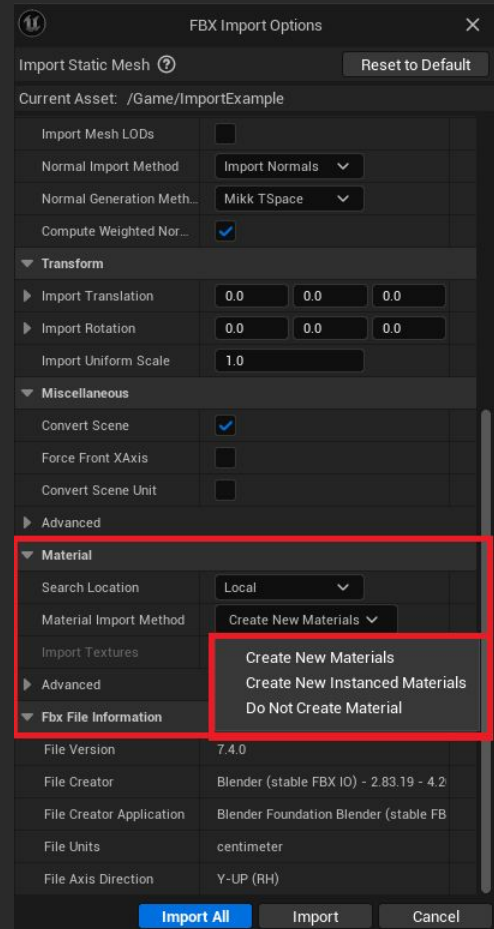
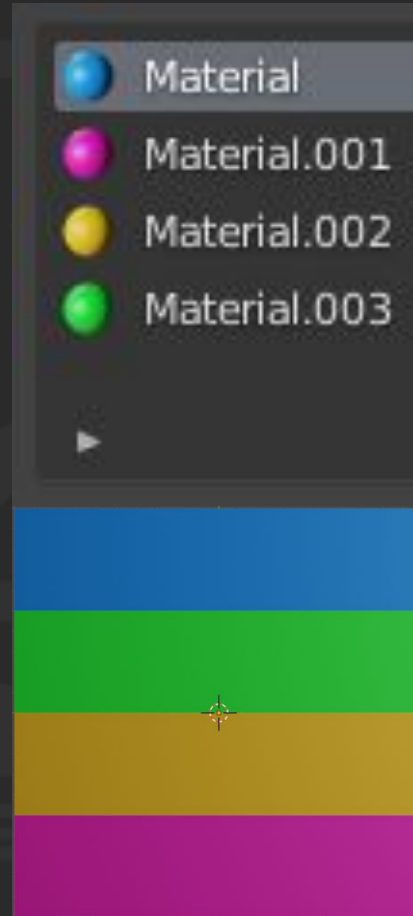
UNREAL
ENGINE

Creating a New material

Content browser

Importing Assets and their materials

Master material vs Material Instances



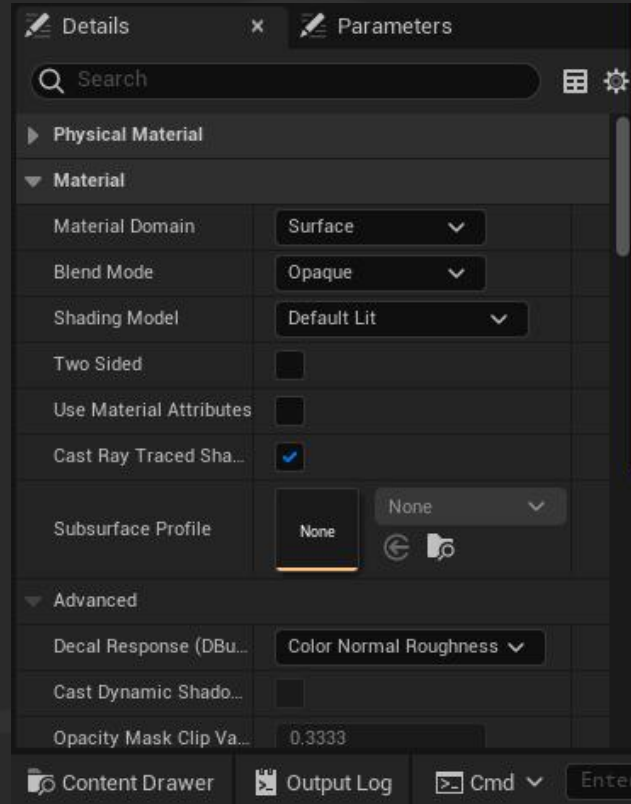
Material Properties

Details Panel

Type of Materials

What is a Blend?

What is a Usage?



Opaque Blend Mode

“It defines a surface through which light neither passes nor penetrates”

Opaque is the most common use

Usually we use it for 90% of the assets



Masked

Works essentially as Math booleans

Mask Opacity becomes enabled when toggled

Works with 0-1 Data to represent the data

When utilizing it, we can make things visible/not visible

Foliage uses this often

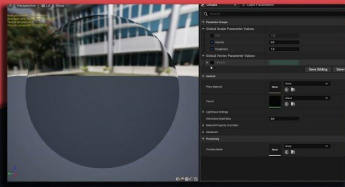
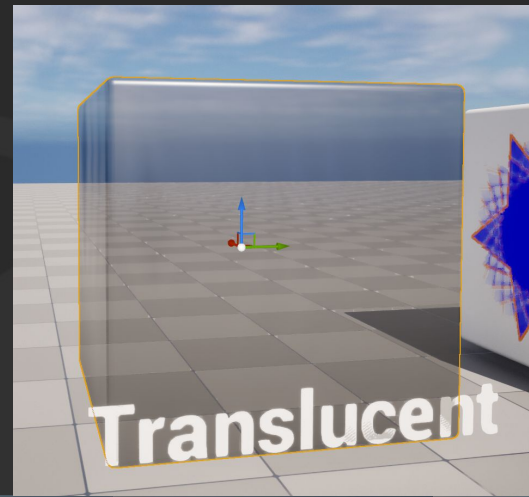


Translucent

In contrast to masked blends,

This is what we utilize to show translucency

0 is the lack of data, vs 1 is the existence of data



REALISTIC GLASS



**UNREAL
ENGINE 5**



Material Expressions

Building blocks of materials

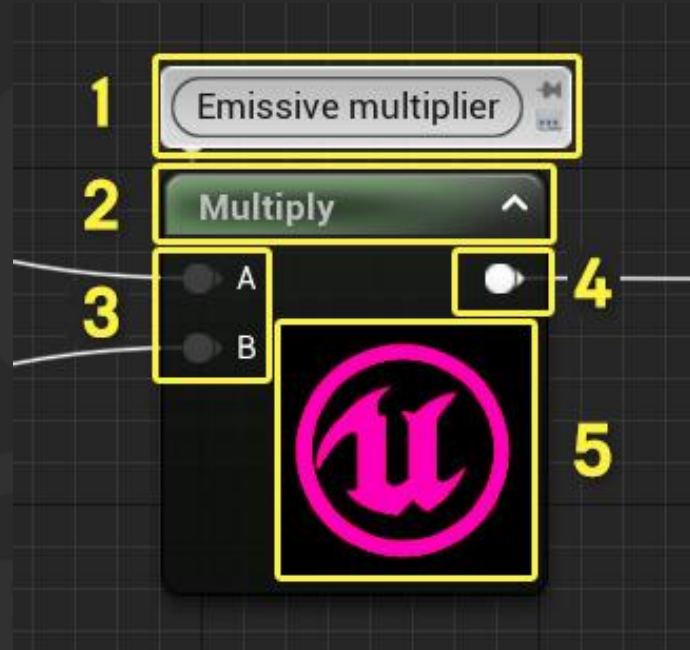
Math and other type of tools to help you create the materials

How to connect things together to make something happen

R G B data and how you can use it

Properties and Values

Making Materials animate



Variables and Parametrization

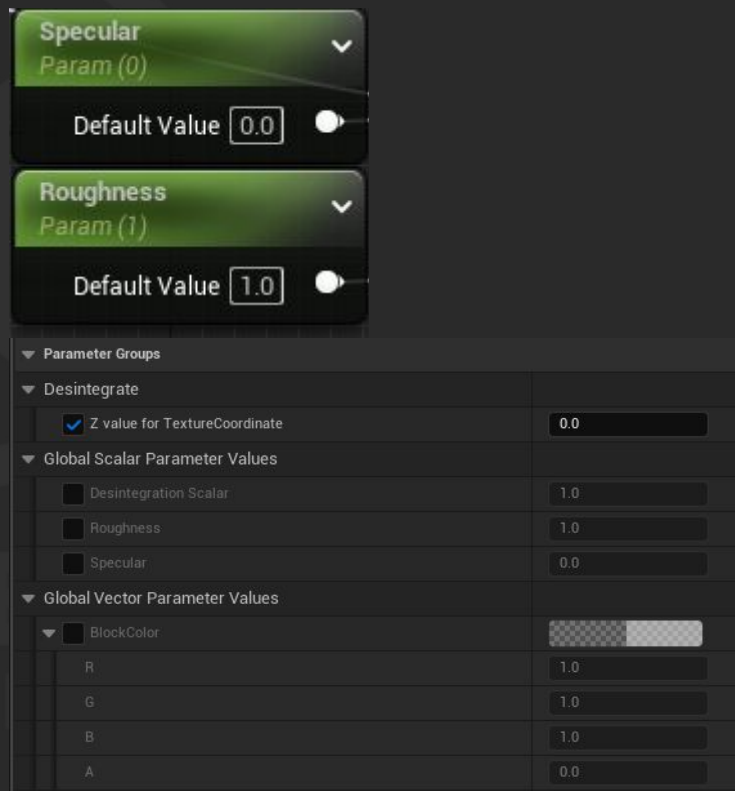
Parameters allow for creating customized versions of the same “code”

Material Instances

Why it's beneficial

Where can you use these parameters?

How do you work with Textures and Other data types?



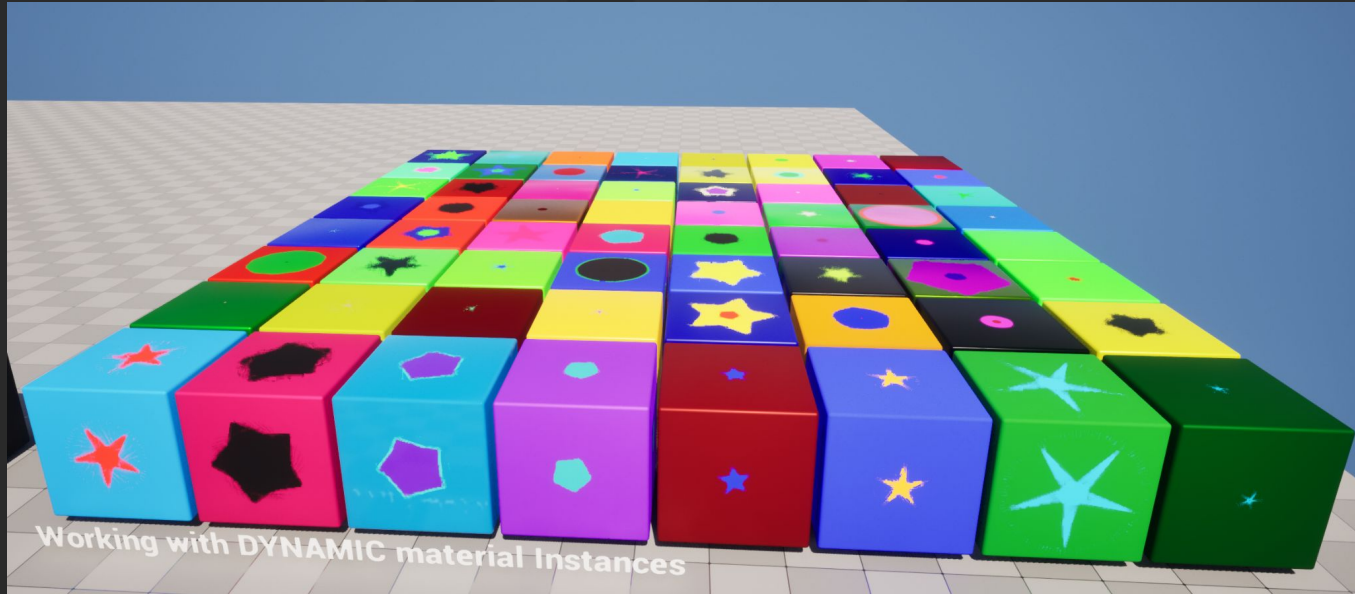
Working with Material Instances

It's important to Understand why we use Material Instances. Static presets of the same code.
Modules?



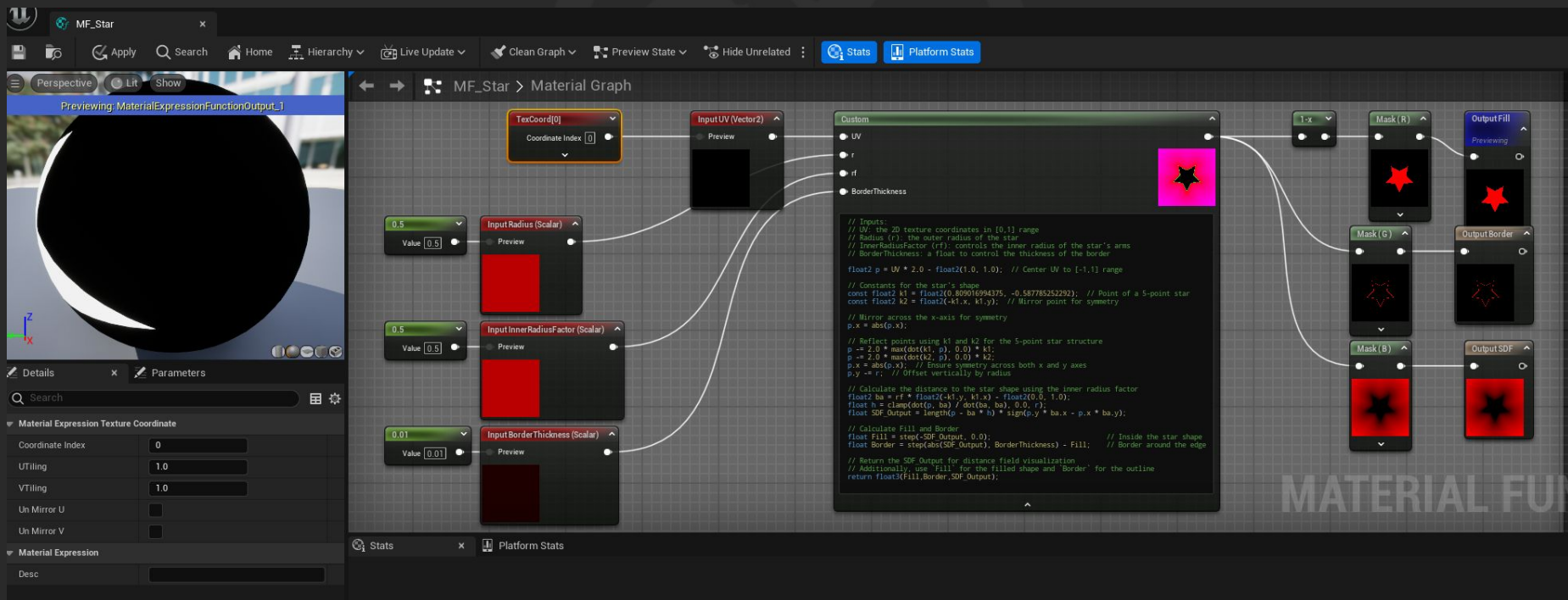
Dynamic Materials

It's important to Understand why we use Material Instances. Dynamic versions of the same code to change during runtime. How do I use them in Blueprint?



HLSL and Material Functions

Diving Deeper into Materials



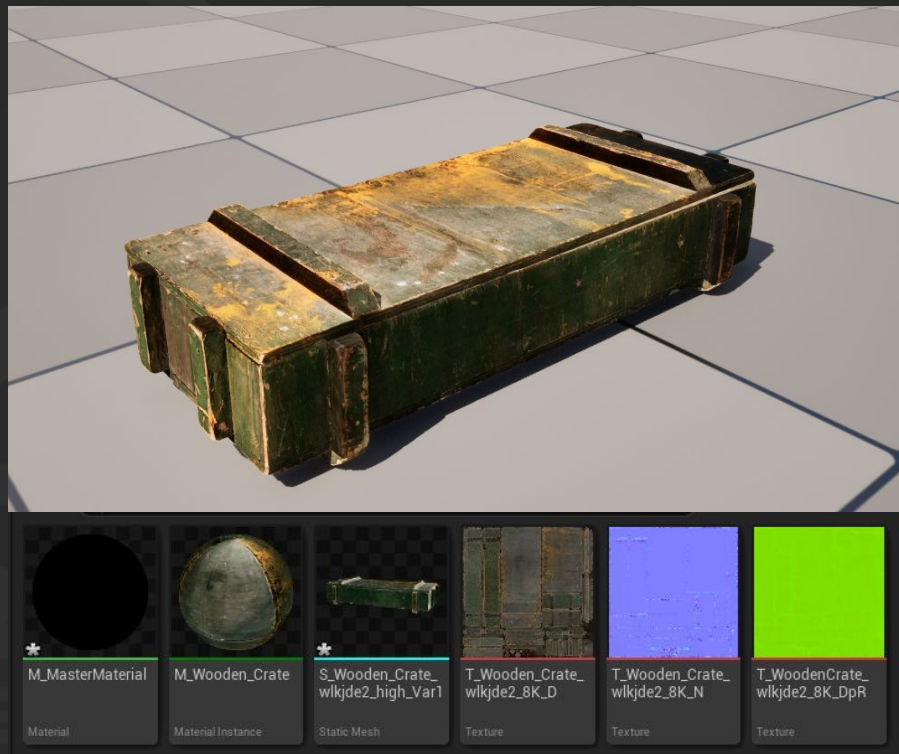
The screenshot displays the Unity Hierarchy and Material Editor. The Hierarchy on the left shows a scene with a sphere and a plane. The Material Editor on the right shows the 'MF_Star' material graph. The graph includes inputs for 'TexCoord[0]', 'InputUV (Vector2)', 'Input Radius (Scalar)', 'Input InnerRadiusFactor (Scalar)', and 'Input BorderThickness (Scalar)'. The 'Custom' node contains HLSL code for a 5-point star. The graph also includes 'Mask (R)', 'Mask (G)', 'Mask (B)', 'Output Fill', 'Output Border', and 'Output SDF' nodes. The 'Output Fill' node is selected, showing a preview of the star shape.

```
// Inputs:  
// UV: the 2D texture coordinates in [0,1] range  
// Radius (r): the outer radius of the star  
// InnerRadiusFactor (rf): controls the inner radius of the star's arms  
// BorderThickness: a float to control the thickness of the border  
  
float2 p = UV * 2.0 - float2(1.0, 1.0); // Center UV to [-1,1] range  
  
// Constants for the star's shape  
const float2 k1 = float2(0.89901994375, -0.587785252292); // Point of a 5-point star  
const float2 k2 = float2(-k1.x, k1.y); // Mirror point for symmetry  
  
// Mirror across the x-axis for symmetry  
p.x = abs(p.x);  
  
// Reflect points using k1 and k2 for the 5-point star structure  
p -= 2.0 * max(dot(k1, p), 0.0) * k1;  
p -= 2.0 * max(dot(k2, p), 0.0) * k2;  
p.x = abs(p.x); // Ensure symmetry across both x and y axes  
p.y -= p.y; // Offset vertically by radius  
  
// Calculate the distance to the star shape using the inner radius factor  
float2 ba = rf * float2(-k1.y, k1.x) - float2(0.0, 1.0);  
float b = clamp(dot(p, ba) / dot(ba, ba), 0.0, r);  
float SDF_Output = length(ba - ba * a) * sign(p.y * ba.x - p.x * ba.y);  
  
// Calculate Fill and Border  
float Fill = step(-SDF_Output, 0.0); // Inside the star shape  
float Border = step(abs(SDF_Output), BorderThickness) - Fill; // Border around the edge  
  
// Return the SDF Output for distance field visualization  
// Additionally, use 'Fill' for the filled shape and 'Border' for the outline  
return float3(Fill, Border, SDF_Output);
```


Deeper Dive

M_lecture will include some most commonly used nodes as a starting point for you to explore with.

Additionally, I've included a example of what a master material can look like and how it is usually setup for projects.



Documentation that I suggest reading

[Getting the Most Out of Noise in UE4 - Unreal Engine](#)

[New shading models and changing the GBuffer | Epic Developer Community](#)

[The Anatomy of an Unreal Engine Master Material - techarthub](#)

One of the most useful tools to understanding the background and optimizations

[Unreal's Rendering Passes](#)

Understanding Texel Density

[Beyond Extent | Texel Density](#)

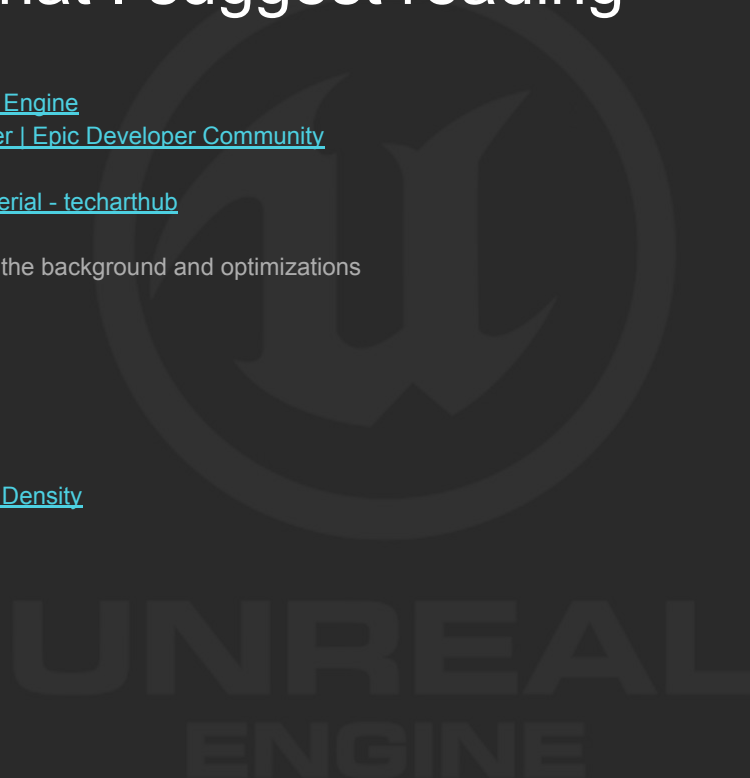
[ArtStation - All You Need to Know about Texel Density](#)

Possibilities of what you can do with Materials

[ArtStation - Space Ship Internal Material](#)

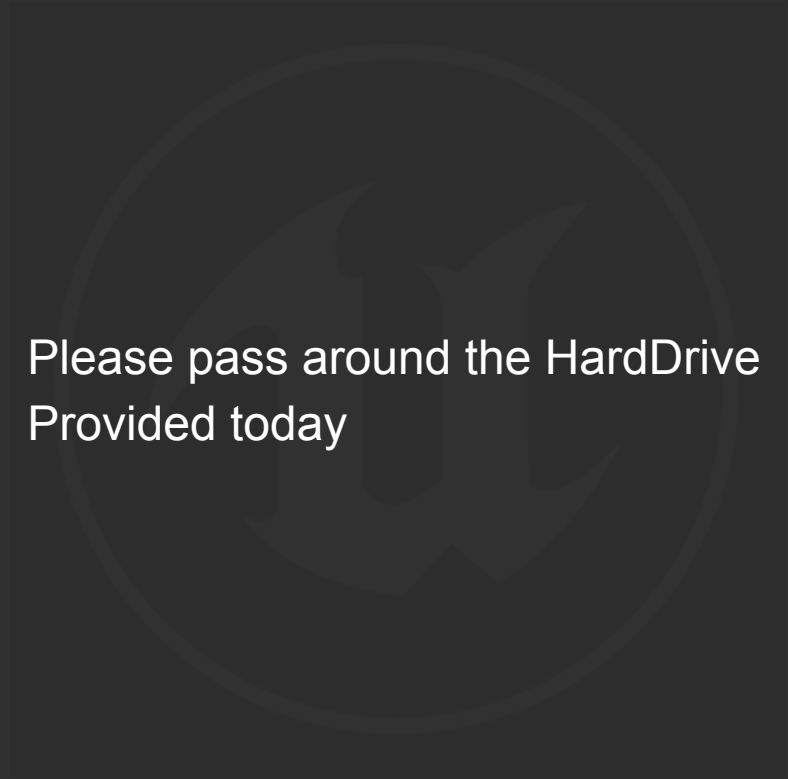
[ArtStation - KSG12 SHG](#)

[ArtStation - Moon / Asteroid Material](#)



3Cs

Andres Ibanez de Aldecoa



Please pass around the HardDrive
Provided today

Integrating Characters to the game

When creating a character we usually adopt a methodology of Rule of 3

What 3 Characteristics can describe the personality?

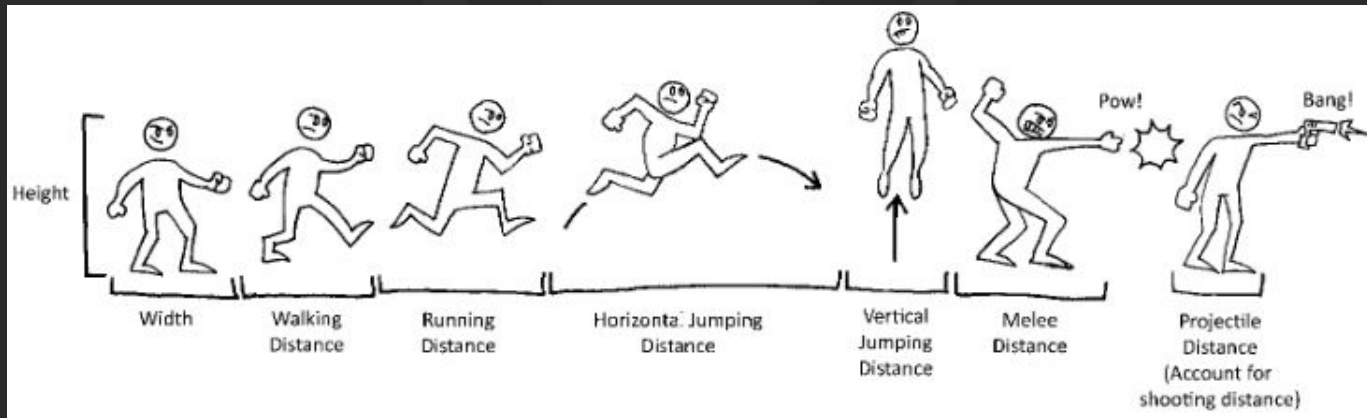
From those, how does the Visual appearance change?

Animators have known for decades that the shapes you use for your character's design will help communicate their personality.



Movement

Movement is if not the most important part of your Character, the most crucial element when designing the game.



Movement Metrics

Height

Width of Passage

Walking Speed

Running Speed

Jump Distance

Jump Height

Melee attack Distance

Projectile Distance



Mechanics Come from Movement Design

Dashing, Dodging, Trampling,
Stomping

How we traverse the environment
and interact with it builds the game.

Charging the weapon, Giving dashes
a cooldown, **stealth**.

The entire game revolves around
Movement



Camera

Static Camera

Parallax Camera

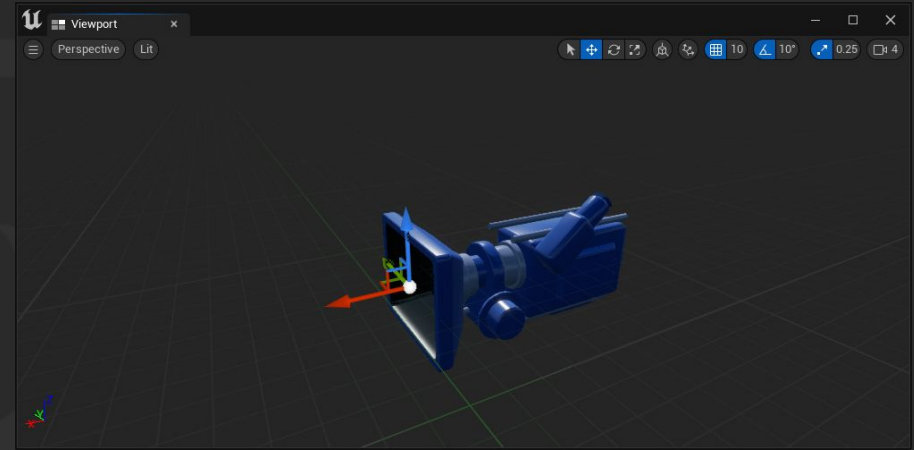
Scrollable Camera

First Person

Third person

Are all influenced by Character design, and movement.

Although we may not notice, Real Life camera work has direct application to how we make games.



Different Control type

Letting the Player Have Control

Deciding to not let players have control

Deciding to let players have control **sometimes**



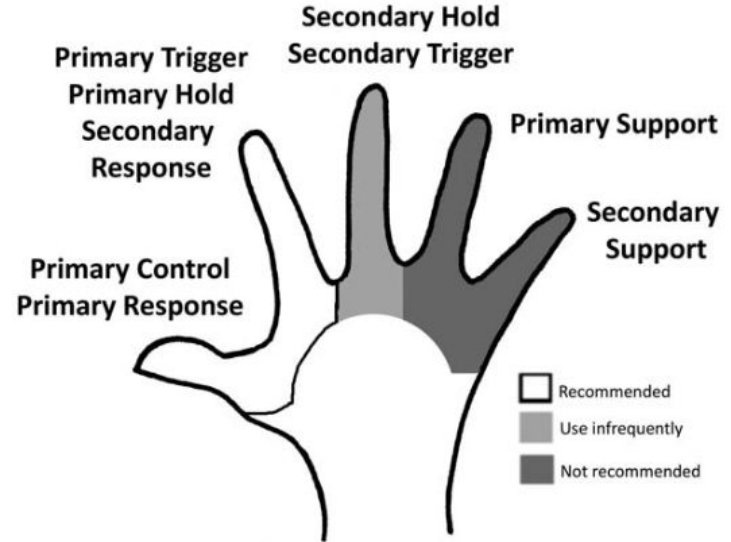
Controls

Every game wants to make its own control scheme. Not all work out...

Users want Customization

Users want Comfort

Muscle Movement



Thumb: flexible with reach. Good for steering and fast response.

Index: strong and fast. Used for response or hold moves.

Middle: weaker but usable for hold moves. Decent reach.

Ring: weak with poor reach. Better for stabilization

Pinkie: poor strength, reach requires hand support.

Traditional, Simplified and Unorthodox Controls

It's fine to want unorthodox control schemes but “traditional” at this point in the game development world exists.

We have muscle memory that has become permanent in our heads.

It's OK to have similar controls to other games

Avoid “repetition” and strive for an uncomplicated control scheme.

Target Audience Control Schemes changes with design



Controls beyond the Input

When considering the interaction of the player and the controls, responsiveness is one of the most important factors

Is the game responsive to you?

Do the animations have Desync?

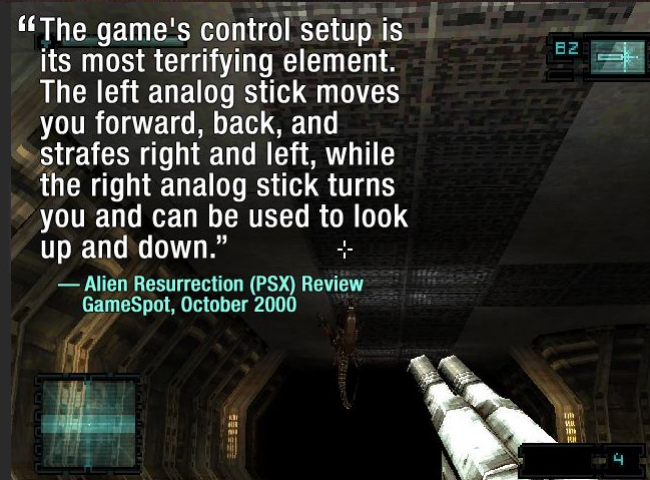
Why can't I control the character I just spent my money on?

Why am I moving the wrong way?



“The game's control setup is its most terrifying element. The left analog stick moves you forward, back, and strafes right and left, while the right analog stick turns you and can be used to look up and down.”

— Alien Resurrection (PSX) Review
GameSpot, October 2000



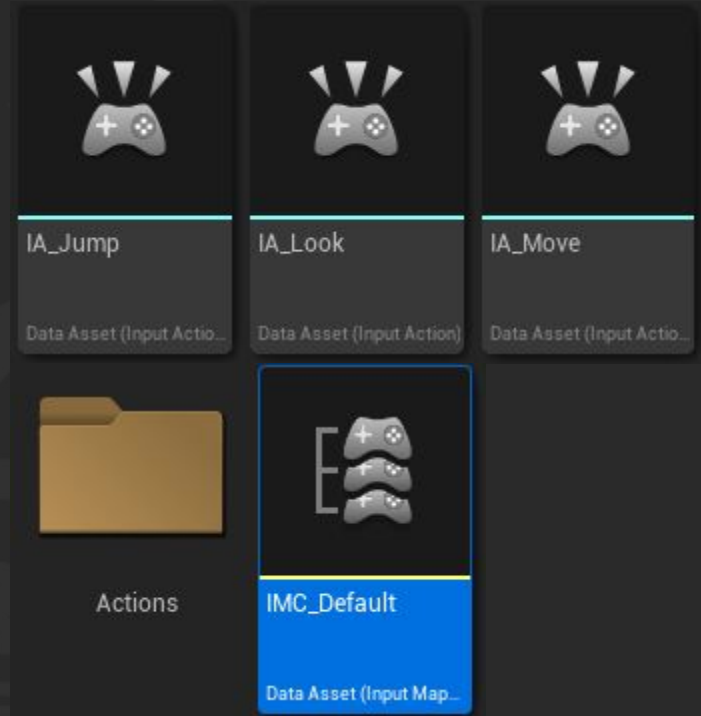
Setup Character Controls

First, We will setup Controls

Data Assets - Input Action

Data Asset - Input Mapping Context

We're going to Make Camera Switches today.



Setting up Camera

Create a Character Class Blueprint

Setting up the Mesh

Adding Cameras

Setting up Multiple cameras and switching between them

First Person and Third Person

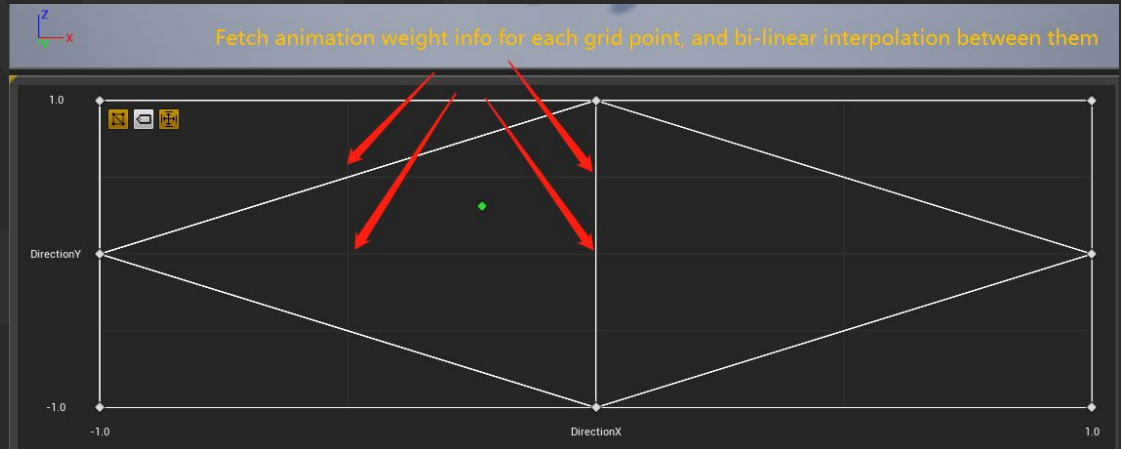


Understanding Character movement

Blendspace - Adding Crouching/ Crouch
Walking

Why Blendspaces are useful

Using transforms as Data



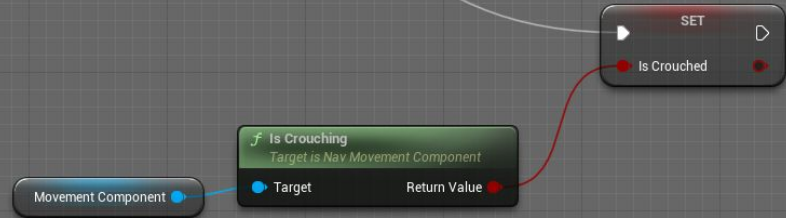
Animation Blueprints

Animation Blueprint Editor

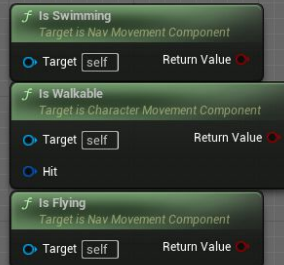
How we will make it so that the character crouches

Setting it up in Character Blueprint

Movement Component already has the crouch feature. We are simply Populating it and making it possible to utilize



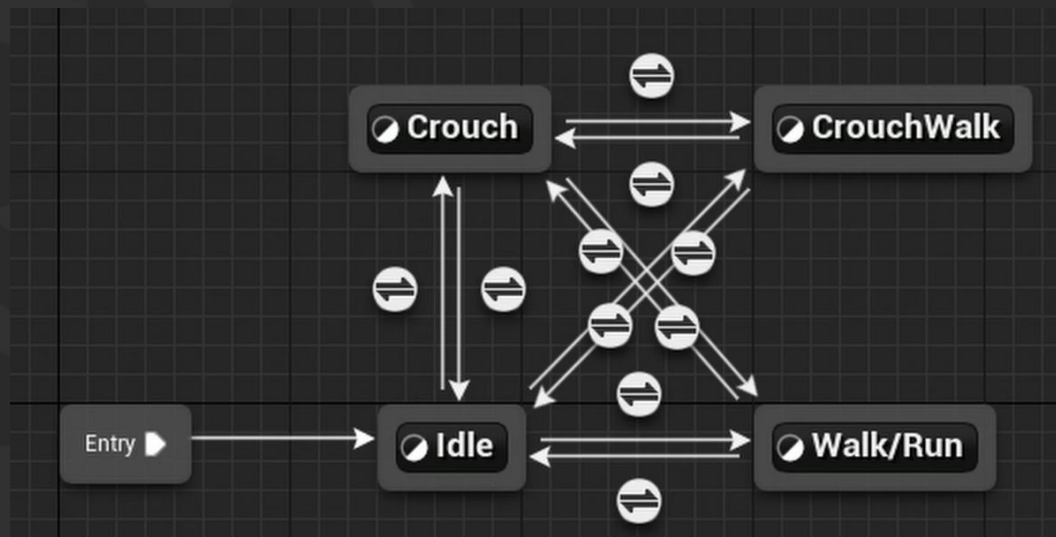
other Options the Movement component has are available for you to mess with. Further options will need to be made yourself



State Alias vs Complex Visual Code

Why we will utilize State Alias

Why creating Complex state machines can become hard to read very quickly



IK RIG

IK RIG- Very useful tool to improve visual quality of Characters.

Allows for positions of feet, hands correctly adapt to environment

Further Studies - Animation with IK Rig is possible

